Design and Evaluation of a RAG Chatbot: Accuracy-Efficiency Trade-offs in Retrieval

CS 541 Artificial Intelligence - Fall 2026 - Stevens Institute of Technology

Team Members Jing Cannon Haiyan Zhu

Project Overview and Motivation

Large language models can generate fluent responses but may produce unsupported answers when they do not have access to relevant domain-specific information. Retrieval-Augmented Generation (RAG) addresses this problem by retrieving relevant documents and providing them to a language model as context before an answer is generated.

We propose a RAG chatbot for financial question answering, with the main research focus on the retrieval layer rather than the user interface. We will study how retrieval method, embedding dimensionality, and retrieval confidence affect accuracy, latency, memory usage, and answer reliability. The retrieval layer will be implemented in PyTorch and benchmarked against FAISS using the FiQA dataset.

Connection to CS 541. The project applies K-nearest-neighbor search and similarity measures, PCA dimensionality reduction, K-means-based approximate indexing, PyTorch implementation, and computational efficiency analysis.

Research Questions

RQ1 - Search algorithm. How do exact and approximate nearest-neighbor methods compare in retrieval quality, latency, and memory usage?

RQ2 - Dimensionality. How does reducing 384-dimensional embeddings to 128, 64, and 32 dimensions with PCA affect retrieval quality and efficiency?

RQ3 - Reliability. How do retrieval depth (top-k) and a similarity threshold affect answer quality and the system's ability to refuse questions that the document collection cannot support?

Related Work

RAG combines retrieval with generation so that answers can be grounded in external evidence (Lewis et al., 2020). Sentence-BERT maps text into semantic vector representations suitable for similarity search (Reimers & Gurevych, 2019). BEIR provides standardized retrieval benchmarks including FiQA and SciFact (Thakur et al., 2021), while FAISS provides exact and approximate vector-search indexes (Johnson et al., 2019). Our contribution is a controlled, course-scale evaluation of retrieval choices and their relationship to RAG reliability.

Dataset

Dataset	Role	Documents	Queries	Labels
FiQA-2018	Main experiments	57,638	6,648 total; 648 test	Query-document relevance
SciFact	Pipeline debugging	5,183	300 test	Query-document relevance

FiQA is large enough to make efficiency differences measurable and includes relevance judgments for retrieval evaluation. SciFact will be used only for early debugging. Retrieval experiments will index whole documents because the benchmark labels are document-level; chunking will be used only for the chatbot demonstration.

For the refusal experiment, we will create a verified unanswerable subset by removing all labeled relevant documents for selected evaluation queries and manually checking that the remaining retrieved documents do not provide the answer.

System Design

1. Encode. Hugging Face *all-MiniLM-L6-v2* converts each document and query into a 384-dimensional vector. Vectors are L2-normalized so inner product corresponds to cosine similarity, and corpus embeddings are cached.

2. Retrieve. We will compare three interchangeable back ends using the same embeddings: (a) our PyTorch brute-force KNN implementation, (b) FAISS IndexFlatIP for exact search, and (c) FAISS IndexIVFFlat for approximate search.

3. Reduce. PCA will be fit on corpus embeddings and used to produce 128-, 64-, and 32-dimensional representations. The same transform will be applied to queries, followed by re-normalization.

4. Decide and generate. The system retrieves the top-k documents. If the best similarity score is below a threshold selected on development data, the system refuses to answer. Otherwise, a fixed small instruction-tuned LLM generates an answer from the retrieved evidence and cites source IDs.

5. Demonstrate. A minimal Streamlit interface will display the generated answer and its supporting sources. The interface is for demonstration; the research contribution is the retrieval and evaluation work.

Experiments and Evaluation

Experiment	Conditions	Primary Measures
E1: Retrieval method	PyTorch brute-force KNN vs. FAISS Flat vs. FAISS IVF. IVF settings are tuned on development data.	Recall@5, Recall@10, MRR@10, nDCG@10, mean/p95 latency, index memory
E2: PCA	384 dimensions vs. PCA to 128, 64, and 32 dimensions using the same search method and k.	Retrieval metrics, latency, memory
E3: Reliability	Top-k = 1, 3, 5, 10 and similarity threshold swept on development data, then fixed for test.	Answer quality, answer rate, correct-refusal rate

All experiments will use the same cached embeddings, query sets, and similarity definition unless the experiment explicitly changes one factor. Hyperparameters such as IVF settings, top-k, and the refusal threshold will be selected using training or development data; final test results will be reported only after settings are fixed.

Generation quality will be evaluated on approximately 100 FiQA test questions using a fixed 1-3 rubric for correctness and groundedness. A sample of 30 responses will also be independently reviewed by both team members. Efficiency measurements will be performed on the same machine using single-query batches, excluding warm-up runs and reporting the mean of repeated runs.

Optional Extension

If E1-E3 are completed on schedule, we will implement a simple K-means-based index that searches only selected clusters and compare it with FAISS IVF. This extension is optional and will not be required for completion of the core project.

Division of Work

Work Package	Jing Cannon	Haiyan Zhu
Data loading, embeddings, unanswerable set	Shared	Shared
E1: PyTorch KNN, FAISS Flat and IVF	Review	Lead
E2: PCA dimensionality reduction	Lead	Review
E3: LLM generation, threshold and refusal	Lead	Review
Evaluation scripts	Shared	Shared
Streamlit demo	Lead	Review
Report and 10-minute presentation	Shared	Shared

Each member will review the complete codebase and must be able to explain the full system, experimental design, and results.

Timeline

Week	Milestone	Deliverable
10	Finalize scope; load data; cache embeddings; verify pipeline on SciFact	Project proposal
11	Run E1 and E2 on FiQA; complete evaluation scripts; prepare unanswerable set	-
12	Analyze E1/E2; run E3; working Streamlit prototype	Midpoint check-in
13	Complete E3; optional extension if time allows; finalize report and video	Report, code, recorded presentation

Risks and Scope Control

If latency differences are small, we will use the full FiQA corpus and report scaling across multiple corpus sizes. Because FiQA relevance labels may be incomplete, proposed unanswerable examples will be manually verified and this limitation will be reported. Generation will be limited to approximately 100 questions to control compute/API cost. If the schedule slips, the optional K-means extension will be dropped first and the Streamlit interface will remain minimal.

Use of AI Tools

Following the course policy, Claude (Anthropic) and ChatGPT (OpenAI) were used to brainstorm project scope and assist with drafting this proposal. The team reviewed and agreed on the project design. Any AI assistance used for code or the final report will also be disclosed.

References

1. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.

2. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

3. Maia, M., et al. (2018). WWW'18 Open Challenge: Financial Opinion Mining and Question Answering (FiQA). Companion Proceedings of The Web Conference.

4. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.

5. Thakur, N., et al. (2021). BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS Datasets and Benchmarks.

6. Wadden, D., et al. (2020). Fact or Fiction: Verifying Scientific Claims. EMNLP.